

NLP in 1 Hour

A Complete Primer

From Text Preprocessing to Large Language Models

What's in this deck



01

NLP Tasks

Word segmentation, POS tagging, NER, classification



02

Text Preprocessing

Tokenization, stemming, stopword removal



03

Text Representation

Bag of Words, TF-IDF, Word Embeddings



04

Deep Learning for NLP

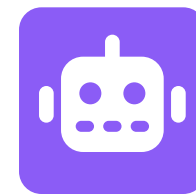
RNN, LSTM, Transformer architecture



05

The Transformer

Attention mechanisms, self-attention, multi-head



06

Large Language Models

GPT, BERT, BART, T5 and pretrained models



07

Beyond NLP

Computer Vision, Multimodal, Text-to-Image



08

Evaluation & Tools

BLEU, ROUGE, ASR, TTS and modern benchmarks

CHAPTER

01 NLP Tasks

What Natural Language Processing can do



The NLP Task Landscape

Word Level	Document Level	Application Level
Word Segmentation — Split text into words/tokens POS Tagging — Label parts of speech Named Entity Recognition — Find names, places, orgs Information Retrieval — Search and extract info	Document Classification — Categorize by topic Spam Detection — Filter unwanted messages Sentiment Analysis — Detect positive/negative tone Fake News Detection — Identify misinformation	Question Answering — Find answers from context Summarization — Condense documents Chatbot — Conversational AI systems Machine Translation — Translate between languages

From token-level analysis to full AI applications — NLP powers the language understanding behind modern software

CHAPTER

02 Text Preprocessing

Cleaning and preparing raw text for models



The Preprocessing Pipeline



Preprocessing is the foundation — garbage in, garbage out. Clean data beats fancy models every time.

CHAPTER

03 Text Representation

Turning words into numbers



From Bag of Words to Embeddings

Method	Bag of Words	TF-IDF	Word Embeddings	Contextual
Approach	Word count vectors	Weighted by rarity	Dense vectors from neural nets	Context-aware vectors
Strength	Simple, fast baseline	Downplays common words	Captures semantic meaning	Same word, different contexts
Weakness	No meaning, sparse	Still no semantics	One vector per word	Computationally heavy
Example	[2, 1, 0, 3]	[0.5, 0.8, 0, 1.2]	king - man + woman $\approx$ queen	BERT embeddings

The evolution: from counting words → weighting words → understanding meaning → understanding context

CHAPTER

04 Deep Learning for NLP

Models that understand language



The 3 Model Families

Encoder-only	Decoder-only	Encoder-Decoder
UNDERSTANDING Models BERT, RoBERTa, DeBERTa WangchanBERTa, ModernBERT Best for Classification, NER, retrieval, embeddings Rule: classify → encoder	GENERATION (AUTOREGRESSIVE) Models GPT-4o, Claude, Llama 3 Qwen2.5, Gemini, DeepSeek Best for Chat, code, reasoning, agents Rule: generate → decoder	SEQ-TO-SEQ Models T5, FLAN-T5, BART mT5, NLLB Best for Translation, summarization, structured output Rule: transform → enc-dec

classify → encoder, generate → decoder, transform → enc-dec

Attention Is All You Need

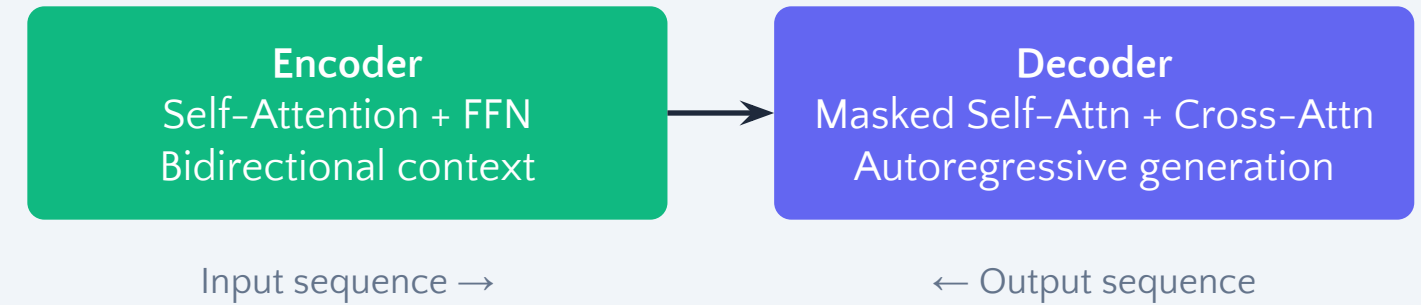
Key Innovation

The Transformer replaces recurrence (RNN/LSTM) entirely with the **attention mechanism**.

This means:

- **More parallelizable** → faster training on GPUs
- **Longer sequences** → better at capturing distant relationships
- **No vanishing gradients** → more stable training

Architecture Overview



Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Q (Query), K (Key), V (Value) are linear projections of the input. The softmax creates a weighted sum of values, where the weights represent how much each position should attend to others. d_k is the dimension of the key vectors.

Four Attention Mechanisms

Self Attention

Q, K, and V all come from the **same input**. Each position attends to all positions in the same sequence.

Used in: Encoder layers — captures bidirectional context

Input $\rightarrow$ Q, K, V $\rightarrow$ Attention $\rightarrow$ Output

Encoder-Decoder Attention

Q comes from the **decoder**, K and V come from the **encoder**. Decoder attends to encoder output.

Used in: Decoder layers — connects input to output

Decoder(Q) + Encoder(K,V) $\rightarrow$ Attention

Masked Self Attention

Prevents the model from seeing **future tokens**. Preserves autoregressive property — can only attend to previous positions.

Used in: Decoder only — model can't see the answer

Mask future positions $\rightarrow -\infty$ before softmax

Multi-Head Attention

Runs multiple attention layers in **parallel** with different projections. Model attends to different representation spaces jointly.

Used in: All Transformer layers — richer representations

$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1 \dots \text{head}_h) W^o$

CHAPTER

05 Large Language Models

The pretrained revolution



The LLM Landscape (2018–2019)

Year	Model	Type	Key Innovation
2018	GPT	Decoder-only	Generative pre-training + fine-tuning paradigm
2018	BERT	Encoder-only	Masked LM + next sentence prediction, bidirectional
2019	RoBERTa	Encoder-only	Optimized BERT: dynamic masking, no NSP, larger batches
2019	DistilBERT	Encoder-only	Knowledge distillation: 40% smaller, 97% performance
2019	ALBERT	Encoder-only	Factorized embeddings, shared params, scales better
2019	BART	Enc-Dec	Denoising autoencoder: BERT's bidirectionality + GPT's generation
2019	T5	Enc-Dec	Text-to-text unified framework: every task as text generation

All Transformer-based. Three families, one architecture — different training objectives create different capabilities.

Choosing the Right Model

Task → Model Mapping

Text Generation (chat, code, stories) → **GPT** models (GPT-4, Claude, Llama)

Seq-to-Seq (translation, summarization) → **BART, T5**

Classification / NER → **RoBERTa, BERT, DeBERTa**

Deployment Options

Closed-source API	Open-weight Hosted	Self-hosted
GPT-4o, Claude, Gemini ✓ Best quality, no infra ✓ Long context ✓ No training needed ✗ \$\$ per token ✗ Data leaves your system ✗ Rate limits	Llama 3, Qwen via Together, Groq ✓ Cheaper than APIs ✓ Faster inference ✓ Less tooling needed ✗ Slightly behind SOTA ✗ Less ecosystem support ✗ Provider dependency	Llama, Qwen, DeepSeek on your GPU ✓ Full control ✓ Data stays on-prem ✓ Custom fine-tuning ✗ GPU costs \$\$\$ ✗ Ops burden ✗ Quantization trade-offs

Prototype fast with APIs → Scale with hosted open-weights → Self-host for privacy/compliance

Beyond NLP

The Transformer architecture powers more than just text

Computer Vision

ViT (Vision Transformer) uses only the encoder architecture from Transformers.

Images are split into patches and treated like tokens in a sequence.

Baselines: ResNet, EfficientNet

SOTA: ViT, Swin Transformer

Text-Image

CLIP learns joint text-image representations from 400M image-text pairs.

Two encoders (text + image) trained with contrastive learning.

Use case: Image search, zero-shot classification

Text-to-Image

Diffusion models generate images from text descriptions using the Transformer backbone.

Models: DALL-E 3, Stable Diffusion, Midjourney

Transformers + Diffusion = photorealistic image generation

The Transformer is a universal architecture — text, images, audio, and beyond.

Evaluation Metrics & Tools

BLEU Score

Bilingual **E**valuation **U**nderstudy — compares translation to reference.

- Measures n-gram precision (1-gram to 4-gram)
- Penalizes short translations (brevity penalty)
- Best for: **Machine Translation**
- Range: 0 to 1 (higher is better)

ROUGE Score

Recall-**O**riented **U**nderstudy for **G**isting **E**valuation — measures overlap.

- Measures n-gram recall (unigram, bigram, LCS)
- Focuses on coverage of reference content
- Best for: **Summarization**
- Range: 0 to 1 (higher is better)

Modern NLP Tools



ASR

Automatic Speech Recognition

Converts spoken language into text. Powers voice assistants, transcription services, and real-time captioning.



TTS

Text-to-Speech

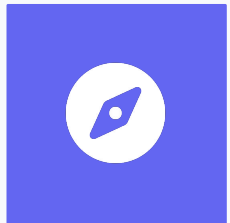
Converts written text into natural-sounding speech. Enables audiobooks, voice assistants, and accessibility tools.

LLM / VLM

for Hackathon

Modern stack: LLMs, fine-tuning, RAG, VLMs, evaluation & deployment

What's in this deck



01

Modern LLM Landscape

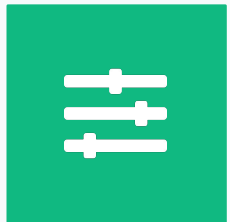
GPT-4o, Claude, Llama 3, Qwen, Gemini, DeepSeek



02

Prompt Engineering

Zero/few-shot, CoT, ReAct, self-consistency



03

Fine-tuning

Full FT vs LoRA / QLoRA / PEFT



04

RAG & Embeddings

Vector DBs, chunking, reranking



05

Vision-Language Models

CLIP, LLaVA, Qwen2-VL, OCR-VLM



06

Inference & Deployment

vLLM, Ollama, quantization



07

Evaluation (modern)

LLM-as-judge, MMLU, HumanEval



08

Thai NLP stack & Kaggle tips

WangchanBERTa, Typhoon, OpenThaiGPT

CHAPTER

Modern LLM Landscape

Where the field is in 2024–2026



The 3 Families (still)

Encoder-only	Decoder-only	Encoder–Decoder
UNDERSTANDING Models BERT · RoBERTa · DeBERTa WangchanBERTa · ModernBERT Best for Classification, NER, retrieval, embeddings	GENERATION (AUTOREGRESSIVE) Models GPT-4o · Claude · Llama 3 Qwen2.5 · Gemini · DeepSeek Best for Chat, code, reasoning, agents	SEQ-TO-SEQ Models T5 · FLAN-T5 · BART mT5 · NLLB Best for Translation, summarization, structured output

Modern LLMs (2023 → 2026)

2023	GPT-4	Llama 2	Claude 2	Mistral 7B	
	Open weights start to compete				
2024	GPT-4o	Claude 3.5	Llama 3	Qwen2	Gemini 1.5
	Multimodal · long context (1M)				
2025	o1 / o3	Claude 3.7	DeepSeek-R1	Llama 3.3	Qwen2.5
	Reasoning models · MoE goes mainstream				
2026	Claude 4.x	GPT-5	Llama 4	Qwen3	Gemini 2.5
	Agentic · cheap inference · on-device				

Choosing an LLM for your task

Closed-source API

GPT-4o, Claude, Gemini

✓ PROS

Best quality
No infra
Long context

✗ CONS

\$\$ per token
Data leaves
Rate limits

USE WHEN

Prototype fast, complex reasoning, no GPU

Open-weight hosted

Llama 3, Qwen2.5 via Together, Groq

✓ PROS

Cheaper
Faster inference
No training

✗ CONS

Slightly behind SOTA
Less tooling

USE WHEN

Cost-sensitive, mid-complexity

Self-hosted

Llama, Qwen, DeepSeek on your GPU

✓ PROS

Full control
Data on-prem
Custom fine-tune

✗ CONS

GPU \$\$\$
Ops burden
Quantize trade-off

USE WHEN

Privacy (med/bank), high volume, fine-tune

CHAPTER

Prompt Engineering

The cheapest way to get a better model



Zero-shot · Few-shot · Chain-of-Thought

Zero-shot

Just ask. No examples.

```
Classify the sentiment of this
review:
"The food was cold and slow."
→ negative
```

Few-shot

Give 2–5 examples first.

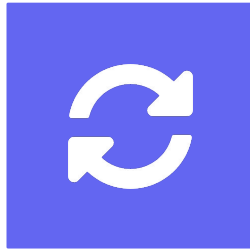
```
Review: "Great food!" → positive
Review: "Slow service." → negative
Review: "Cold pasta." →
```

Chain-of-Thought

Ask the model to reason step-by-step.

```
Q: A bat & ball cost $110. Bat is
$100 more than ball. How much is
the ball?
Let's think step by step...
→ $5
```

Advanced: ReAct - Self-consistency - Structured output



ReAct

Reason + Act. Model alternates between thinking and calling tools.

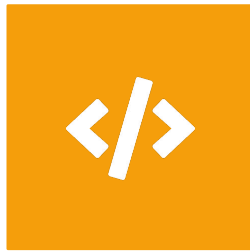
→ *Agents, RAG with iterative search*



Self-consistency

Sample N answers, take majority vote. Trades cost for accuracy.

→ *Math, multi-step reasoning*



Structured output

Force JSON / schema with response_format or grammar.

→ *Field extraction (e.g. your Kaggle OCR)*



Role prompting

"You are an expert radiologist..." — primes style and vocab.

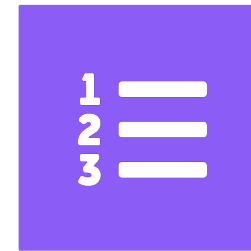
→ *Medical, legal, code review*



Guardrails

Constrain output with regex, schemas, or output classifiers.

→ *Production, safety-critical*

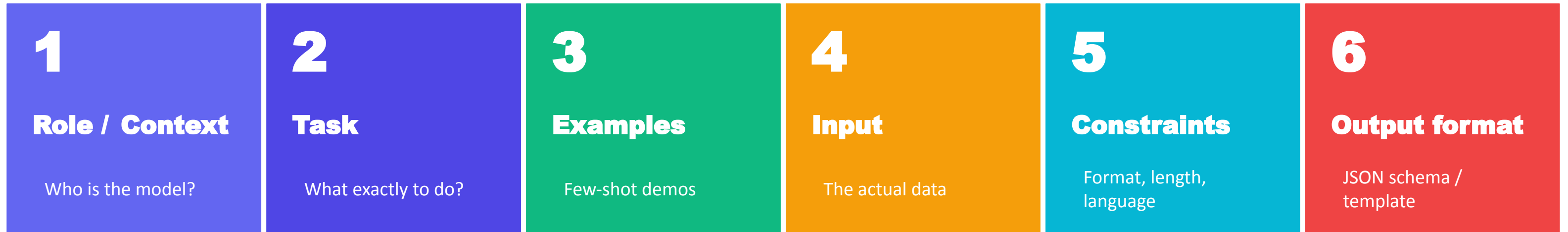


Decomposition

Break big task → sub-prompts → combine. (Least-to-most prompting)

→ *Long docs, complex pipelines*

Anatomy of a great prompt



```
# Role
You are a Thai medical NER expert.

# Task
Extract drug names, dosages, and frequencies from clinical notes.

# Examples
Input: "ให้ paracetamol 500mg q4h" → {"drug":"paracetamol","dose":"500mg","freq":"q4h"}

# Input
{{clinical_note}}

# Output
Return ONLY valid JSON matching the schema above.
```

CHAPTER

Fine-tuning

When prompts aren't enough



The customization spectrum

Prompt eng.	RAG	LoRA / QLoRA	Full fine-tune	Pre-train
COST Free DATA NEEDED 0 Try this first, always	COST \$ DATA NEEDED Docs Add knowledge without training	COST \$\$ DATA NEEDED 100s–10k Adapt style, format, domain	COST \$\$\$ DATA NEEDED 10k+ Heavy domain shift	COST \$\$\$\$\$ DATA NEEDED Billions Almost never. Don't.

LoRA: train tiny adapters, not the whole model

Idea

Freeze the original weights W . Add two small matrices $A \cdot B$ (rank $r \ll d$) and only train those.

$$h = W \cdot x + (B \cdot A) \cdot x$$

Where $W \in \mathbb{R}^{(d \times d)}$, $A \in \mathbb{R}^{(r \times d)}$, $B \in \mathbb{R}^{(d \times r)}$, $r \approx 8-64$

0.1–1%

of params trained

10–100×

less GPU memory

MBs

adapter file size

= or >

perf. vs full FT

```
from peft import LoraConfig, get_peft_model

cfg = LoraConfig(
    r=16, lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05, bias="none")
model = get_peft_model(base, cfg)
# trainable: ~0.3% of params ✨
```


EVEN CHEAPER

QLoRA & the PEFT family

QLoRA = LoRA + 4-bit quantized base model. Fine-tune 70B on a single 48GB GPU.

LoRA

~10GB / 7B model

Trainable low-rank matrices

QLoRA

~6GB / 7B model

LoRA + NF4 quantization of base

DoRA

Similar to LoRA, better perf

Decomposed magnitude + direction

Prefix tuning

Tiny – KBs

Trainable prefix tokens only

Prompt tuning

Tiny – KBs

Soft prompt embeddings

IA³

Tiny – 0.01%

Scale activations with learned vectors

CHAPTER

RAG & Embeddings

Give the model knowledge it doesn't have



Retrieval-Augmented Generation



Why RAG?



CHOOSE YOUR STORE

Vector databases



DB	Hosting	Best for	Scale
 FAISS	Library	Fast prototyping, in-memory	1M+
 Chroma	Local / server	Hackathon-friendly, simple API	1M
 Qdrant	Self-host / cloud	Production filters, hybrid search	100M+
 Weaviate	Self-host / cloud	Schema-rich, GraphQL	100M+
 pgvector	Postgres ext.	Already on Postgres? Use this	10M
 Milvus	Self-host / cloud	Massive scale, GPU-accel	1B+

 *Tip: For Thai content, test BGE-M3 or multilingual-e5-large embeddings first.*

Things that actually move the needle

Chunk size

300–800 tokens with 10–20% overlap. Sentence-aware splits beat fixed length.

Hybrid search

BM25 + dense vectors → combine with RRF. ~10% recall boost.

Reranker

Top-50 from vector → rerank to top-5 with cross-encoder (bge-reranker).

Metadata filters

Filter by date, source, language BEFORE retrieval. Massive precision win.

Query rewriting

Use LLM to expand/clarify the query, or HyDE (hypothetical answer first).

Multi-query

Generate 3 paraphrases → retrieve for each → union. Great for vague questions.

CHAPTER

Vision-Language Models

When text isn't the whole story



Vision-Language Models

A VLM takes both images and text as input. Output is text — captions, answers, JSON, code.



Tasks: captioning · VQA · document understanding · OCR · UI grounding · diagram reasoning

Modern VLMs you should know

CLIP 2021	Image-text contrastive. Zero-shot classification, retrieval. → Search, filter	BLIP-2 2023	Q-Former bridges frozen vision + frozen LLM. → Caption, VQA
LLaVA 2023	Simple projection from CLIP → Vicuna/Llama. Open & easy. → VQA, chat	Qwen2.5-VL 2024-25	Native dynamic resolution. Strong on Thai + docs. → OCR, docs, agents
InternVL 2024	Open SOTA. Up to 78B. Beats GPT-4V on some. → Research SOTA	Closed VLMs 2024+	GPT-4o · Claude · Gemini. Easiest path to high quality. → Production prototype

Document & OCR VLMs

Use case: read a document image → output structured JSON (your Kaggle OCR challenge).

Traditional OCR + LLM

- 1. OCR engine (Tesseract, EasyOCR, PaddleOCR, Surya)
- 2. LLM parses text → JSON
- 3. Post-process & validate

✓ Modular • Cheap • On-prem

✗ OCR errors cascade • Bad on layout

End-to-end VLM

- 1. Feed image directly to VLM
- 2. Prompt with target JSON schema
- 3. Parse output

✓ Layout-aware • Robust to noise • Less code

✗ Slow • Pricey • Hallucinates fields

OCR-specialized: Surya • Typhoon OCR • Donut • Nougat • GOT-OCR2 • Qwen2.5-VL • olmOCR

CHAPTER

Inference & Deployment

Making LLMs fast enough to be useful



Pick the right inference engine

Ollama Go Local dev, demos, Mac users	llama.cpp C++ On-device, CPU/Metal, embedded
vLLM Python/CUDA Server-side high throughput, PagedAttention	TGI (HF) Rust/Python HF stack, multi-GPU serving
TensorRT-LLM C++/CUDA Max speed on NVIDIA hardware	SGLang Python Complex prompting, structured outputs

Hackathon default: Ollama for local + vLLM for serving + OpenAI/Anthropic API for max quality.

Quantization: trade quality for speed

Reduce precision of weights (and sometimes activations) to fit bigger models on smaller GPUs.

FP16 / BF16	INT8	INT4 (NF4, AWQ)	INT3 / INT2
MEMORY 100% <i>Baseline</i>	MEMORY 50% <i>≈ 99% quality</i>	MEMORY 25% <i>≈ 95–98% quality</i>	MEMORY <20% <i>Quality drops fast</i>

Popular formats

GGUF (llama.cpp)	AWQ (vLLM-friendly)
GPTQ (older, widely supported)	BitsAndBytes (training)

CHAPTER

Evaluation (modern)

BLEU/ROUGE are not enough



THE PROBLEM

BLEU & ROUGE are not enough

These n-gram metrics were designed for translation/summarization with reference text. Modern LLMs generate diverse, long, open-ended outputs.

Example: clinical SOAP note generation (your TMS-Scribe finding)

Reference

*Patient reports 3-day cough. No fever. Lungs clear.
Dx: viral URI. Plan: rest, fluids.*

Model A

*3-day cough, afebrile, clear lung exam.
Likely viral URI. Supportive care.*

ROUGE-1: 0.31 · Clinical: ★★★★★

Model B

*Patient reports cough. No fever. Lungs clear.
Viral. Rest.*

ROUGE-1: 0.67 · Clinical: ★★★★★

ROUGE rewards copying; it punishes paraphrase and good structure.

The modern evaluation toolbox

LLM-as-judge

Open-ended generation

GPT-4 / Claude scores outputs vs rubric or pairs.

💡 Use 2 judges for reliability.

Pass@k / unit tests

Code (HumanEval, MBPP)

Run generated code against test cases.

💡 $k=1$ for prod, $k=10$ for research.

Exact match / F1

QA, NER, extraction

Token or span-level overlap.

💡 Normalize whitespace, case.

MMLU / BIG-Bench

General knowledge benchmark

Multiple-choice across domains.

💡 Public; train sets may leak.

Embedding similarity

Semantic similarity

BERTScore, cosine of embedded outputs.

💡 Better than ROUGE for paraphrase.

Human eval (gold)

Clinical, legal, safety

Domain experts grade samples.

💡 Expensive — sample 50–200.

CHAPTER

Thai NLP & Kaggle

tips

Local stack + competition strategy



Thai NLP modern stack

Tokenization & basics

- PyThaiNLP — Swiss-army knife
- deepcut — DL word seg
- newmm — fast dict-based
- ssg — sentence segmenter

Encoder models

- WangchanBERTa (VISTEC)
- Pathumma-LLM-BERT
- PhayaThaiBERT
- XLM-R multilingual

Thai LLMs

- Typhoon (SCB-10X) ★
- OpenThaiGPT
- Pathumma-LLM
- SeaLLM (regional)

ASR / TTS / OCR

- Typhoon-ASR-Realtime
- Pathumma-TTS
- Typhoon-OCR v1.5
- Surya (multilingual)

WIN MORE

Kaggle NLP playbook

1

Read the eval metric first

BLEU? Exact match? Custom? Your whole pipeline depends on it.

2

Build a strong baseline fast

Day 1: simple pipeline submitting end-to-end. Iterate from there.

3

Trust local CV, not LB

Public LB overfits. Build a robust CV that mirrors the test distribution.

4

Ensemble at the end

Avg of 3–5 diverse models (different seeds/folds/architectures)
→ free boost.

5

Data > model

Clean labels, augment, find leaks. A clean dataset beats a fancy model.

6

Time-box experiments

Set a budget per idea. Kill bad ones fast. Document everything.

Hackathon checklist

- Define the task precisely** — Classification? Generation? Extraction? Multi-modal?
- Sample the data** — Look at 50 raw examples. Spot edge cases before they bite.
- Pick a baseline model** — BERT/roBERTa for classification · llama/qwen for gen · CLIP/Qwen-VL for VL.
- Track everything** — wandb / mlflow / spreadsheet. Future you will thank you.
- Know the eval metric** — What does the leaderboard score? Optimize that, not proxies.
- Set up validation** — Stratified K-fold or time-split. Match test distribution.
- Build the submission pipeline first** — End-to-end, even if dumb. Submit on Day 1.
- Plan for failure** — Backup plan if GPU dies, dataset breaks, deadline shifts.

ESSENTIAL HACKATHON STACK

Models: transformers · peft · trl · unsloth · sentence-transformers
Data: datasets · pandas · polars · pyarrow
RAG: langchain / llamaindex · chromadb / qdrant · bge-m3
Infer: vllm · ollama · llama.cpp · openai / anthropic SDK
Eval: evaluate · ragas · lm-eval-harness · llm-as-judge scripts
Thai: pythainlp · wangchanberta · typhoon · openthaigpt

Where to learn more

Hugging Face

docs · model hub · datasets · spaces

huggingface.co

Unsloth Notebooks

Free fine-tuning notebooks, very fast

unsloth.ai

PyThaiNLP

Thai NLP library — start here for Thai

pythainlp.org

Papers with Code

SOTA benchmarks per task

paperswithcode.com

Karpathy lectures

Build GPT from scratch · zero-to-hero

[youtube](#) → [@AndrejKarpathy](#)

Illustrated Transformer

Jay Alammar — the classic visual intro

jalammar.github.io

Kaggle NLP comps

Past solutions write-ups are gold

kaggle.com/competitions

lm-evaluation-harness

Standard eval suite

[github](#) → [EleutherAI](#)

Go build

The best way to learn NLP is to ship something this week.

→ Pick a Kaggle dataset · Build a baseline · Iterate · Submit

Questions?